Title : Comparative Performance Evaluation of Sparse, Dense and Hybrid Retrieval for Product Search on Amazon ESCI

Students Name	Registration No.	Email-id
Tanishq Choudhary	23FE10CSE00664	tanishqchoudhary007@gmail.com
Chinta Sri Durga Rohit	23FE10CSE00596	rohitch.5616@gmail.com
Ishan Ravishankar	23FE10CSE00641	ishanravishankar.15@gmail.com

Abstract — This report compares four ranking methods for e-commerce product search on a 1,000-query sample of the Amazon Shopping Queries (ESCI) dataset. Two of them are lexical, TF-IDF and Okapi BM25, and both run on an inverted index implemented for this project. The third is a dense retriever based on the all-MiniLM-L6-v2 sentence transformer, and the fourth combines BM25 with the dense retriever through Reciprocal Rank Fusion (RRF). All parameters were tuned on 300 development queries. The models were then evaluated once on the remaining 700 queries under two protocols: re-ranking the judged candidates of each query, and retrieval from the full catalog of 19,876 products. In re-ranking the four methods are separated by only 0.008 nDCG@10, although all of them are well above a random ordering. In full-catalog retrieval the differences are larger. BM25 reaches 0.570 nDCG@10, the dense retriever 0.556 and TF-IDF 0.527, while the hybrid reaches 0.600 nDCG@10 and 0.860 Recall@100 and is significantly better than every single model (Wilcoxon signed-rank test, $p < 0.001$). A per-query analysis shows that the dense retriever tends to fail on colours, sizes and rare brand names, whereas BM25 fails on misspellings and compound words. These errors rarely coincide, which explains the gain from fusion. A web interface was also built that displays per-term score contributions and the ESCI judgment of each result.

Keywords: *Information Retrieval · Product Search · Amazon ESCI · BM25 · TF-IDF · Sentence Transformers · Reciprocal Rank Fusion*

1. Introduction

Most online shops are entered through a search box, and the ranking behind that box decides which products a customer sees at all. Product search differs from web search in several respects. Queries are short, often three or four words such as “wood handle induction pan”. The documents are product titles of about fifteen words, written by sellers who repeat popular terms to attract traffic. Relevance is also stricter. A pan with a plastic handle is about the right subject, yet a shopper who asked for a wooden handle would regard it as a wrong result.

Two families of retrieval models are commonly used for this task. Lexical models score a product by the query terms that occur in its title. TF-IDF [5] weights each matching term by its frequency in the title and its rarity in the catalog. Okapi BM25 [1] adds a saturating function of term frequency and a normalisation for title length. These models are fast and need no training data, and their scores can be explained term by term. Their main weakness is vocabulary mismatch. A title that says “earphones” does not match a query for “earbuds”. Dense models address this by encoding the query and the title as vectors with a neural network, so that texts with similar meaning lie close together [2]. The cost is a network evaluation for every query and, as Section 4 shows, a weaker grip on exact attributes such as sizes, colours and model numbers.

Because the two families make different errors, combining them is attractive. Linear interpolation of scores requires the scores to be calibrated against each other, which is awkward when one model produces BM25 values around 15 and the other produces cosine similarities between 0 and 1. Reciprocal Rank Fusion (RRF) [3] avoids the problem by using only the rank of each document in each list. It has a single constant and no trainable weights, which makes it well suited to a small project without labelled training data.

The Amazon Shopping Queries Dataset [4] makes it possible to study these questions on real customer queries. Each query comes with candidate products judged on a four-level scale (Exact, Substitute, Complement, Irrelevant), so ranking quality can be measured with graded metrics rather than a single relevant or irrelevant decision.

The objective of this project was to implement TF-IDF, BM25, a sentence-transformer retriever and an RRF hybrid on a shared preprocessing pipeline and inverted index, and to compare them under one evaluation setup. Two questions guided the analysis. The first was where the dense model outperforms BM25 and where it falls behind. The second was whether running both models and fusing their rankings is worth the additional cost per query. A search interface was built alongside the experiments so that individual rankings could be inspected directly.

The study is limited to the English part of the dataset and to product titles. Japanese and Spanish queries would require their own tokenizers, stop-word lists and stemmers, and a multilingual encoder for the dense model, which was outside the time available. All results come from a single fixed split of the queries, so the differences reported in Section 4 are supported by significance tests on the test queries rather than by repeated splits.

2. About Dataset

2.1 Visualization of dataset in form of graphs or tables and explanation of dataset using an example

The Shopping Queries Dataset was released by Amazon for the KDD Cup 2022 [4]. The full release contains about 130,000 queries in English, Japanese and Spanish, each with up to 40 candidate products retrieved by Amazon's production search. Annotators assigned every query-product pair to one of four classes. *Exact* means the product matches the query in all its specifications. *Substitute* means it does not match fully but could serve the same purpose. *Complement* means it would be used together with the requested item, such as a case for a phone. *Irrelevant* covers everything else. The dataset defines three tasks, of which Task 1 (ranking the candidates of a query) is the one closest to ours.

Our sample consists of 1,000 English queries from the test split of the reduced version of the dataset, together with all of their 20,156 judgments. These cover 19,876 distinct products, of which 267 were judged for more than one query. For each product the sample provides the title and, for 94% of products, the brand. Descriptions and bullet points from the full release are not included, so all models in this study work on titles only. Table 1 summarises the sample.

Table 1. Statistics of the evaluation sample.

Property	Value
Queries (dev / test)	1,000 (300 / 700)
Judged query-product pairs	20,156
Distinct products	19,876
Judged products per query (min / median / mean / max)	8 / 16 / 20.2 / 85
Words per query (mean / median / max)	3.95 / 4 / 20
Words per title (mean / median / max)	17.1 / 16 / 78
Queries containing a digit	22%
Queries whose candidates include all four labels	12.6%
Labels E / S / C / I	8,692 / 7,234 / 836 / 3,394

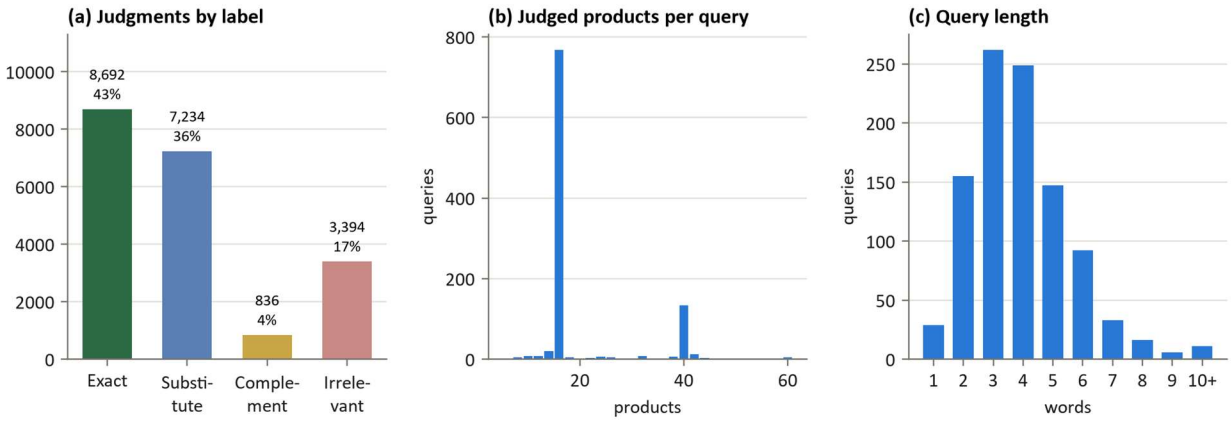


Figure 1: Label distribution, judged products per query and query length in the sample.

Figure 1a shows that Exact and Substitute together account for 79% of the judged pairs. This has a direct consequence for evaluation. If both classes were treated as relevant, almost any ordering of the candidates would place about eight relevant products in the top ten, and precision at ten would say little about the ranking. The first version of our evaluation made this choice and reported P@10 of about 0.84 for every model. Binary metrics in this report therefore count only Exact as relevant, while nDCG uses all four grades. Figure 1b shows that most queries have 16 judged candidates and a smaller group has 40, so labels run out quickly when a model ranks the whole catalog. Figure 1c shows that queries are short, with a mean of 3.95 words.

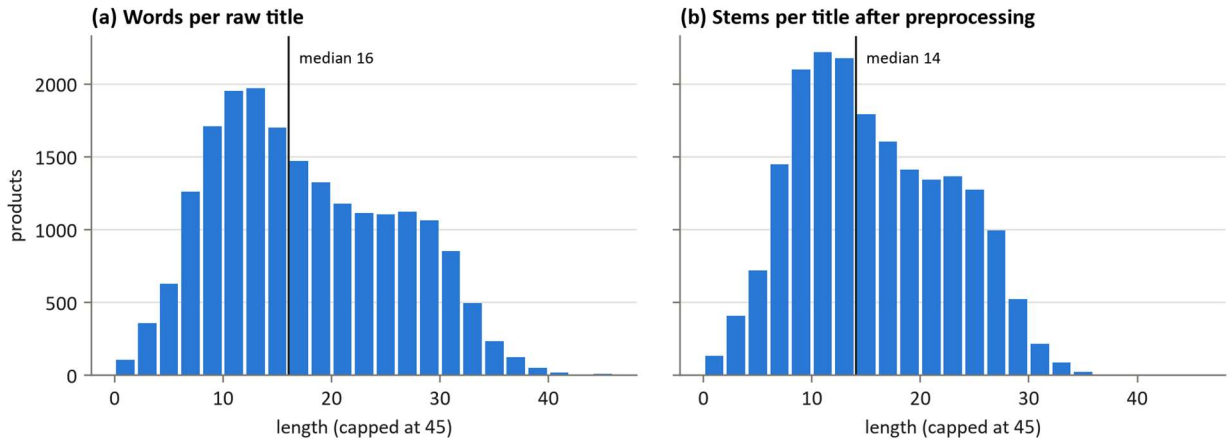


Figure 2: Title length before and after preprocessing.

Titles are much longer than queries (Figure 2). The median raw title has 16 words and the longest has 78. For 25% of products the brand did not appear in the title and was appended to the indexed text. About 22% of queries contain a digit, usually a size, a quantity or a model number, which is why the tokenizer described in Section 3.1 keeps alphanumeric tokens together.

Table 2. Judged products for the query “turbo dive minion” (one or two per label).

Label	Product title
Exact	WowWee Minion MiP Turbo Dave - Fun Balancing Robot Toy
Exact	Turbo Dave
Substitute	Minions Plush Buddies -Stuart, 5 Inches
Complement	Universal Minions Despicable Me 3 R/C Inflatable Mini Minion Mel Remote Controlled...
Irrelevant	kidrobot- Futurama Universe X Random Mini Figure Blind Box 2.5"
Irrelevant	Turbo

Table 2 illustrates the labels with one query. The query asks for a toy version of Dave, a character from the Minions films. Robot toys of Dave are Exact. A plush figure of a different Minion is a Substitute, and a remote-controlled Minion doll is a Complement. The candidate list also contains products such as a DVD titled “Turbo”, which shares a query word but is Irrelevant. A lexical model would score that title highly, and cases of this kind recur in the error analysis of Section 4.6.

3. Methodology

Figure 3 gives an overview of the system. In an offline step the product titles are preprocessed and stored in an inverted index, and the same titles are encoded into an embedding matrix. At query time the query passes through the same preprocessing, each model scores the catalog, and the hybrid fuses the BM25 and dense rankings. The web interface and the evaluation scripts call the same ranking code, so a result seen in the interface is the result that was evaluated.

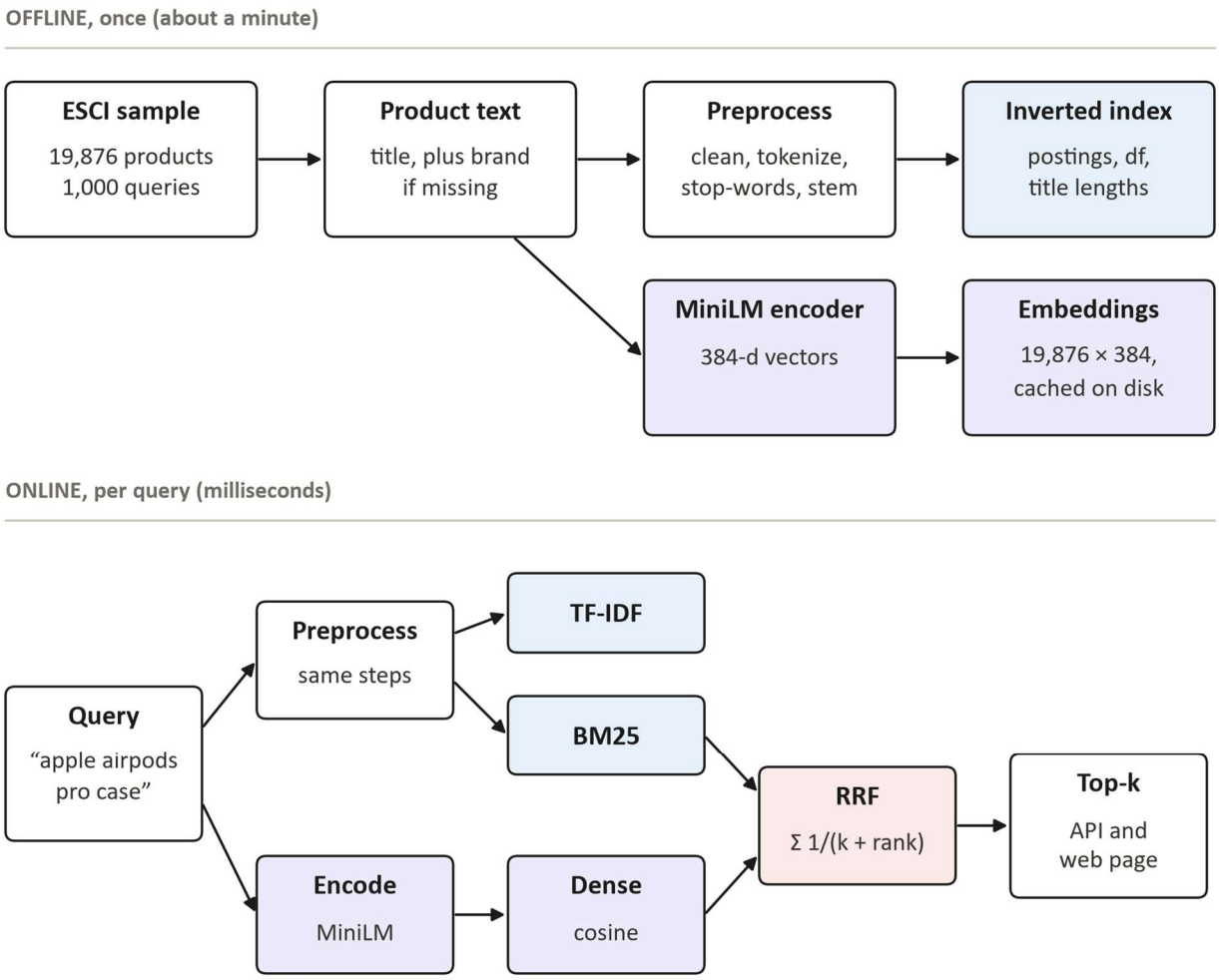


Figure 3: Architecture of the retrieval system. Blue marks the sparse path, violet the dense path and red the fusion step.

3.1 Preprocessing and Indexing

Each product is represented by its title, with the brand appended when the title does not already contain it. Figure 4 follows one title through the four preprocessing steps. First, HTML entities are decoded, tags and URLs are removed, Unicode is normalised and the text is lower-cased. Second, a regular-expression tokenizer splits the text into words while keeping internal hyphens and decimal points, so that “128gb”, “usb-c” and “8.5” each remain one token. Third, tokens shorter than two characters and the 198 words of the NLTK English stop-word list are removed. Fourth, the Porter

stemmer [6] reduces each remaining token to its stem, for example “running” to “run” and “batteries” to “batteri”. The example also exposes one weakness that we left in place. The possessive in “Women’s” becomes a separate token “s”, which is then discarded as too short.

The stems are stored in an inverted index implemented in Python with NumPy. The index is built in a single pass over the catalog. For each title the stems are counted, and the pair (document id, term frequency) is appended to the postings list of every stem. Because documents are visited in order, each postings list is already sorted by document id when the pass ends, and it is then stored as two NumPy arrays. The index also keeps the document frequency of each stem, the length of every title in stems and the average length. Over our catalog it holds 30,010 distinct stems and 274,924 postings, with an average of 15.2 stems per title.

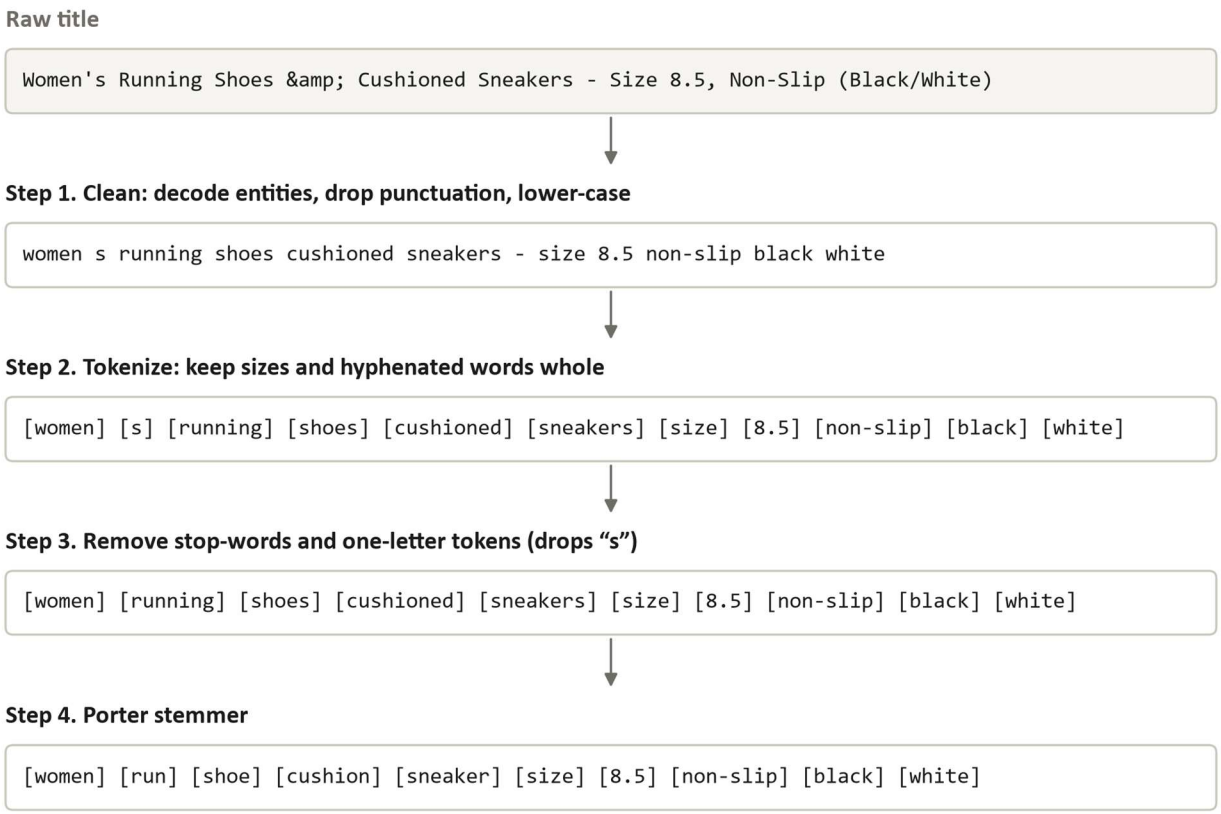


Figure 4: The four preprocessing steps applied to a product title (actual output of the code).

Queries are scored term at a time. An accumulator array with one entry per product is first set to zero. For each query stem the model then reads its postings list and adds the stem’s contribution to the entries of the listed products. Only products that share at least one stem with the query are touched, and the work grows with the total length of the postings lists involved, not with the size of the catalog. Scoring all 19,876 products takes about a third of a millisecond. Figure 5 shows the structure on a catalog of three titles, and Figure 6 the statistics of the full index. Document frequency follows the usual

long-tailed distribution: 57% of stems occur in a single title, while the most frequent stem, “black”, occurs in 2,654.

Catalog

d1	Apple AirPods Pro Case, Silicone Cover
d2	AirPods Pro (2nd Generation) Wireless Earbuds
d3	Silicone Case for iPhone 13 Pro

Stem	Postings (doc : tf)	df	idf
airpod	d1:1 d2:1	2	0.470
pro	d1:1 d2:1 d3:1	3	0.134
case	d1:1 d3:1	2	0.470
silicon	d1:1 d3:1	2	0.470
appl	d1:1	1	0.981
earbud	d2:1	1	0.981

BM25, “airpods pro case”

1. d1 1.048
2. d3 0.634
3. d2 0.589

k1 = 1.2, b = 0.75
average length 5.67
only the postings of the query stems are read

Figure 5: Postings lists, document frequency and idf for a three-product catalog, and the BM25 ranking they produce (computed by our code).

Document ids and term frequencies are stored as 32-bit numbers, so the 274,924 postings occupy about 2.2 MB and the whole index fits easily in memory. Building it takes 6.6 s on the laptop described in Section 4.1.

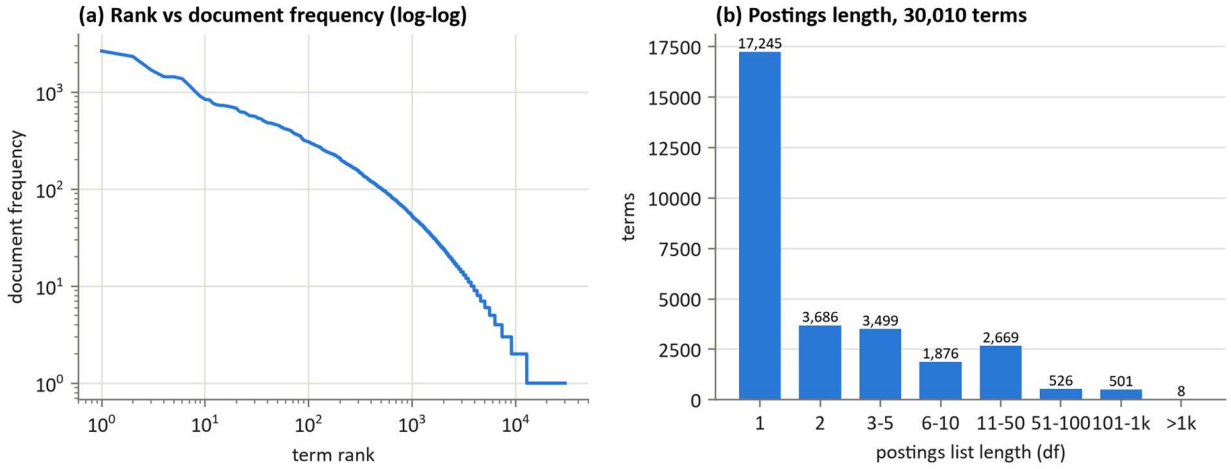


Figure 6: Statistics of the inverted index built over the sample catalog.

3.2 Ranking Models

Each of the four models returns a score for every product in the catalog. The same three-product catalog from Figure 5 is used as a running example, with the query “airpods pro case”; the resulting scores are listed in Table 3.

- **TF-IDF vector space model:** a term t in title d receives the weight w(t,d) = (1 + ln f(t,d)) · idf(t), where f(t,d) is the term frequency and idf(t) = ln((1 + N)/(1 + df(t))) + 1 for a catalog of N titles. The score of a title is the cosine between its weight vector and that of the query. In the implementation the contributions are accumulated over the postings and divided by title norms computed when the index is built. The unit tests compare our scores with scikit-learn’s TfidfVectorizer, and they agree to within 10^{-9}.
- **Okapi BM25:** the score of a title is the sum over query terms of idf(t) multiplied by a term-frequency factor, as given in Equation 1. We use the idf of Equation 2, which is the form used in Lucene. The original Robertson idf becomes negative for stems that occur in more than half the titles, and a negative weight would penalise a title for containing a query word. The parameters k1 and b were selected on the development queries (Section 4.3), giving k1 = 0.6 and b = 1.0.

$$\text{score}(q, d) = \Sigma\ (t \in q)\ \ \text{idf}(t) \cdot f(t,d)(k1 + 1)\ /\ (\ f(t,d) + k1(1 - b + b \cdot |d|/\text{avgdl})\ ) \qquad (1)$$

$$\text{idf}(t) = \ln(\ 1 + (N - \text{df}(t) + 0.5)\ /\ (\text{df}(t) + 0.5)\ ) \qquad (2)$$

- **Sentence Transformer:** the all-MiniLM-L6-v2 model [7] maps a text to a 384-dimensional vector by mean pooling over the token embeddings of a small transformer. The model was trained with a contrastive objective on more than one billion sentence pairs, and inputs longer than 256 word pieces are truncated, which no title in our catalog reaches. Titles are cleaned but neither stemmed nor stripped of stop-words, since the model was trained on ordinary text. All vectors are L2-normalised, so the dot product equals the cosine similarity. Encoding the catalog took about a minute and a half on a laptop CPU, and the matrix is cached on disk.
- **Hybrid RRF:** the hybrid takes the BM25 ranking and the dense ranking, each cut at three times the requested depth with a minimum of 50, and fuses them with Equation 3 (Section 3.3).

Table 3. Running example: scores of the three products for “airpods pro case” (BM25 with k1 = 1.2, b = 0.75 as in Figure 5; RRF with k = 10).

	Title	TF-IDF	BM25	Dense	RRF
d1	Apple AirPods Pro Case, Silicone Cover	0.607	1.048	0.771	0.1818
d2	AirPods Pro (2nd Generation) Wireless Earbuds	0.340	0.589	0.653	0.1603
d3	Silicone Case for iPhone 13 Pro	0.404	0.634	0.443	0.1603

In Table 3 all three models put d1 first, since it contains every query term. The lexical models then prefer d3, which matches “case” and “pro”, while the dense model prefers d2, which is the AirPods themselves and is closer in meaning. In BM25 the stem “pro” contributes only 0.130 to d1 against 0.459 for “airpod”, because “pro” occurs in all three titles and has a low idf. RRF combines the two orderings and, in this small case, gives d2 and d3 the same score, since each is second in one list and third in the other.

Figure 7 shows the role of the two BM25 parameters. With b = 0, the term-frequency factor rises from 1 for a single occurrence towards k1 + 1 as occurrences increase, so k1 controls how quickly repetition stops adding to the score. The parameter b scales the penalty for titles longer than average. Our setting of k1 = 0.6 saturates early, and b = 1.0 applies full length normalisation.

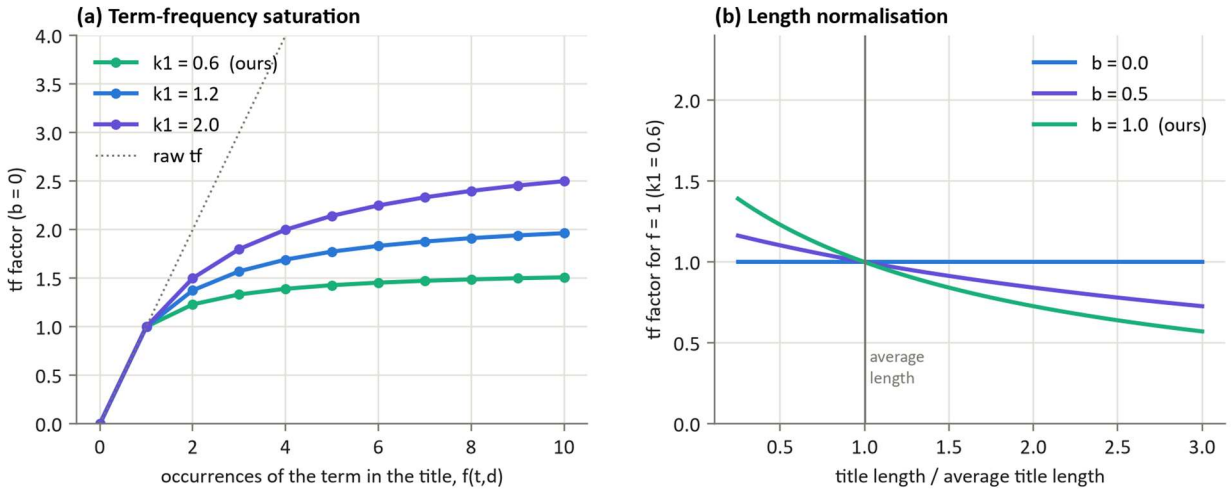


Figure 7: BM25 term-frequency factor as a function of (a) term occurrences and (b) relative title length.

Figure 8 gives an impression of the dense representation. It projects the embeddings of the Exact and Substitute products of four test queries onto their first two principal components. Products judged for the same query form compact groups, and each query vector lies inside or near its own group. This is the property the dense model relies on. It does not need the query words to appear in the title, only that the title is close in meaning. The separation is not perfect. A few products lie between the groups, and a projection onto two dimensions discards most of the information in the 384 original ones, so the figure should be read as an illustration and not as a measurement.

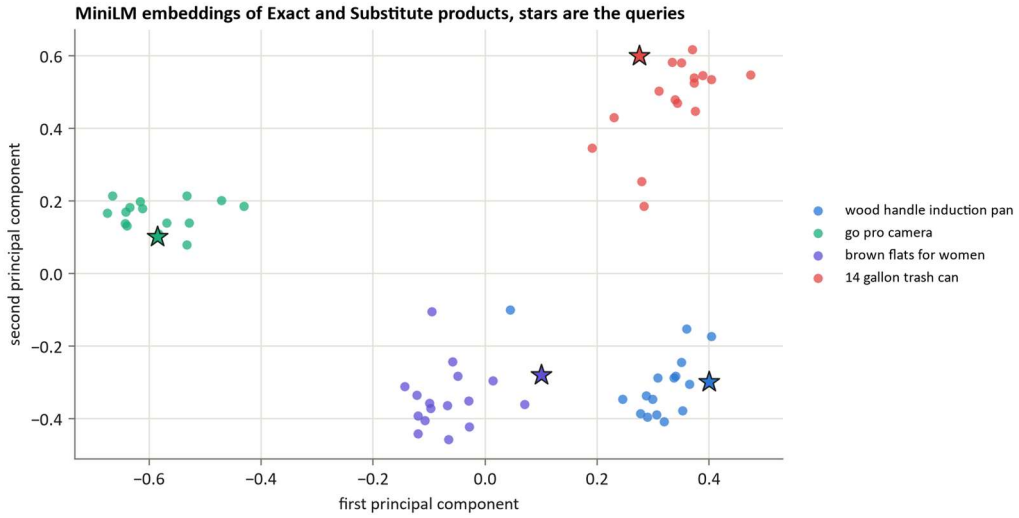


Figure 8: Two-dimensional PCA projection of MiniLM embeddings for four test queries.

3.3 Reciprocal Rank Fusion Formulation

Let M be a set of rankers and k a constant. RRF scores a document by summing, over every list that contains it, the reciprocal of its rank shifted by k:

$$\mathrm{RRF}(d) = \Sigma\ (m \in M)\ \ 1\ /\ (k + r_m(d)) \qquad (3)$$

Here $r_m(d)$ is the 1-based rank of d in the list of model m. With k = 10, a product ranked first by both models scores 2/11 = 0.182. A product ranked first by one model and absent from the other scores 0.091, which is less than the 0.143 of a product ranked fourth by both. The method therefore rewards agreement between two models that work in very different ways. A smaller k concentrates the weight on the top positions of each list. Cormack et al. used k = 60 [3], and on our data the value made little difference (Section 4.5). Figure 9 shows the fusion on a real query from the catalog.

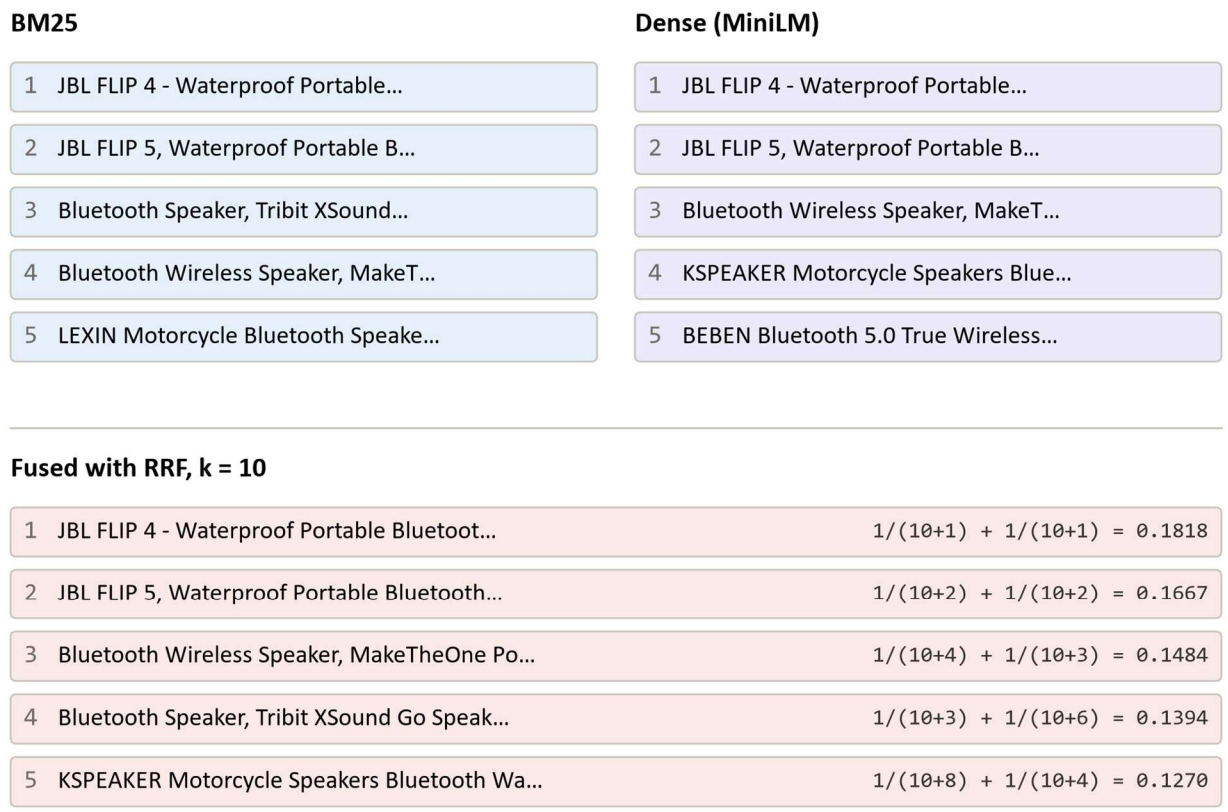


Figure 9: Top five of BM25, of the dense model and of the fused list for one query, with the RRF arithmetic for each fused product.

3.4 Evaluation Protocol and Metrics

The 1,000 queries were split once, with a fixed random seed, into 300 development and 700 test queries (Figure 10). All parameter choices and design decisions were made on the development queries, using the best nDCG@10 averaged over the two protocols. The test queries were scored a single time, for

Section 4. In the *re-ranking* protocol, which corresponds to Task 1 of the KDD Cup, a model orders only the candidates judged for the query. In the *retrieval* protocol it ranks the whole catalog, and products that were never judged for the query count as not relevant. Retrieval scores are therefore a lower bound on the true quality.

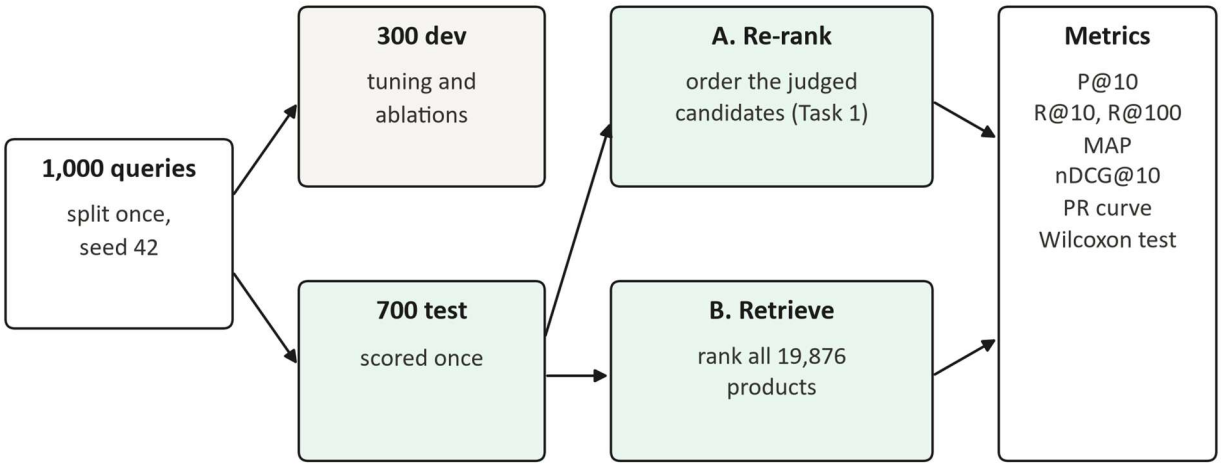


Figure 10: Query split and the two evaluation protocols.

Before re-ranking, each candidate list is shuffled with a fixed seed. BM25 often assigns the same score, frequently zero, to several candidates, and a stable sort keeps tied items in their input order. If the stored order carried information about the labels, it would leak into the metrics. A random ordering of the candidates is reported as a baseline for the same reason.

The metrics are defined below for a ranked list and the set R of Exact products of a query. Here rel(i) is 1 if the product at rank i is Exact and 0 otherwise, and g(i) is its graded gain (E 3, S 2, C 1, I 0). The definition of nDCG follows Järvelin and Kekäläinen [8].

$$\mathrm{P@k} = (1/\mathrm{k})\ \Sigma\ (\mathrm{i} \le \mathrm{k})\ \mathrm{rel(i)} \qquad \mathrm{R@k} = (1/|\mathrm{R}|)\ \Sigma\ (\mathrm{i} \le \mathrm{k})\ \mathrm{rel(i)} \qquad (4)$$

$$\mathrm{AP} = (1/|\mathrm{R}|)\ \Sigma\ (\mathrm{i})\ \mathrm{rel(i)} \cdot \mathrm{P@i} \qquad \mathrm{MAP} = \text{mean of AP over queries} \qquad (5)$$

$$\mathrm{DCG@k} = \Sigma\ (\mathrm{i} \le \mathrm{k})\ (2\text{^}\mathrm{g(i)} - 1)\ /\ \log_2(\mathrm{i} + 1) \qquad \mathrm{nDCG@k} = \mathrm{DCG@k}\ /\ \mathrm{IDCG@k} \qquad (6)$$

IDCG@k is the DCG of the ideal ordering of the judged products. We report P@10, Recall@10, Recall@100 (for retrieval only, since every judged candidate lies within the top 100 when re-ranking), MAP and nDCG@10, together with 11-point interpolated precision-recall curves [9]. Two models are compared with a two-sided Wilcoxon signed-rank test on per-query nDCG@10 [10].

As a worked example, consider BM25 over the full catalog for “wood handle induction pan”. The labels of its top ten are E, E, S, E, -, S, E, S, E, E, where a dash marks a product that was not judged for this query. The query has 16 judged products, 12 of them Exact. Six of the top ten are Exact, so P@10 = 0.6

and R@10 = 0.50. Average precision over the top 100 is 0.597. The DCG of the top ten is 24.41, while an ideal list of ten Exact products would reach 31.80, giving nDCG@10 = 0.767.

The same example shows the effect of unjudged products. The fifth result has no label and is counted as not relevant, although its title describes a cast-iron pizza pan with handles that some shoppers would accept. Cases like this are the reason retrieval scores are treated as lower bounds.

3.5 System Implementation

The models are served by a FastAPI application, and the interface is a single page written in HTML, CSS and JavaScript without a build step. Figure 11 traces one search request. The backend builds all models once at start-up from the cached catalog and embeddings, which takes about ten seconds, and then answers each request from memory. Table 4 lists the endpoints.

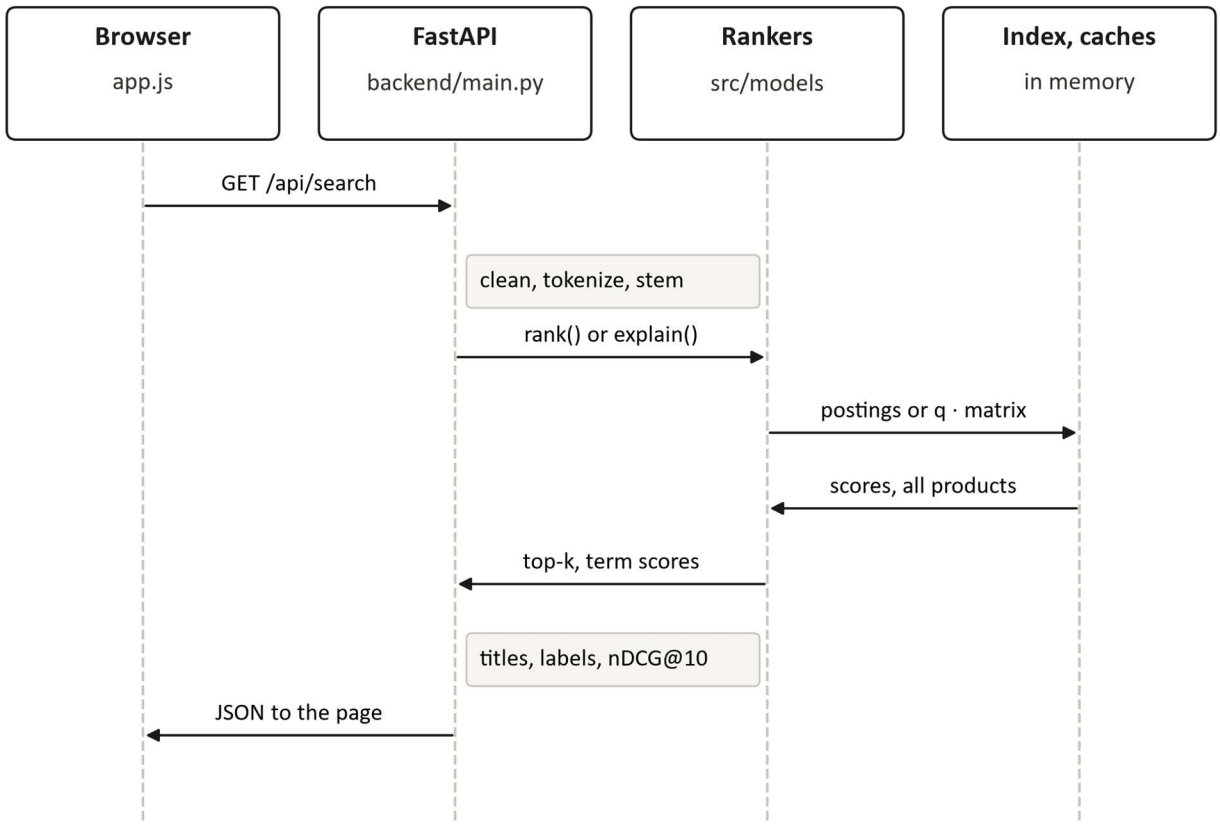


Figure 11: Path of a search request through the web application.

Table 4. Endpoints of the backend.

Endpoint	Returns
GET /api/search	ranked products for one model, preprocessing trace, per-term scores, labels, nDCG@10
GET /api/compare	the top ten of all four models for the same query
GET /api/queries	a random sample of benchmark queries for the start page
GET /api/evaluation	test tables, precision-recall curves and the BM25 grid
GET /api/index	index size, most frequent stems and label counts

4. Experimental Setup and Performance Analysis

4.1 Setup

All experiments ran on a laptop with an Intel Core i7-13620H processor and 16 GB of memory, using the CPU only. Table 5 lists the settings of each model. The complete evaluation, including the parameter searches on the development queries, takes about four minutes once the catalog embeddings are cached.

Table 5. Model settings used in all test experiments.

Model	Setting
Preprocessing	regex tokenizer, NLTK stop-words (198), Porter stemmer
TF-IDF	logarithmic tf, smoothed idf, cosine similarity
BM25	k1 = 0.6, b = 1.0, Lucene idf
Sentence Transformer	all-MiniLM-L6-v2, 384 dimensions, cosine similarity
Hybrid	RRF of BM25 and the dense model, k = 10
Software	Python 3.11, NumPy 2.4, scikit-learn 1.8, PyTorch 2.14 (CPU), sentence-transformers 6.2, FastAPI 0.157

4.2 Main results

Table 6 gives the re-ranking results and Table 7 the full-catalog results on the 700 test queries. The best value in each column is shown in bold.

Table 6. Re-ranking the judged candidates (ESCI Task 1), test queries.

Ranking model	P@10	R@10	MAP	nDCG@10
TF-IDF	0.4874	0.6636	0.6274	0.7781
BM25	0.4876	0.6642	0.6385	0.7806
Sentence Transformer	0.4869	0.6555	0.6342	**0.7856**
Hybrid (RRF)	**0.4901**	**0.6663**	**0.6393**	0.7856
Random order	0.4401	0.5750	0.5173	0.6962

Table 7. Retrieval from all 19,876 products, test queries.

Ranking model	P@10	R@10	R@100	MAP	nDCG@10
TF-IDF	0.3216	0.4400	0.7963	0.3952	0.5268
BM25	0.3456	0.4711	0.8064	0.4357	0.5704
Sentence Transformer	0.3386	0.4620	0.8230	0.4166	0.5559
Hybrid (RRF)	**0.3631**	**0.4956**	**0.8604**	**0.4627**	**0.5997**

In re-ranking, every model performs well above the random baseline, which reaches 0.696 nDCG@10. The four models, however, finish close together. TF-IDF is lowest at 0.778 and the dense model highest at 0.786, a difference of 0.008. None of the pairwise differences among TF-IDF, BM25 and the dense model is significant (all $p > 0.20$, Table 8). The candidates of each query were retrieved by Amazon's own

search engine for that query, so most of them already share the main query words. What remains is largely the distinction between Exact and Substitute, and a title alone often does not contain enough information to make it.

Retrieval from the full catalog separates the models far more. BM25 is ahead of TF-IDF by 0.044 nDCG@10, and most of this gap comes from length normalisation. The cosine measure of TF-IDF favours very short titles, and for “wood handle induction pan” its first result is a book titled “In the Woods”. The dense model lies between the two lexical models in nDCG@10 but has a higher Recall@100 than BM25 (0.823 against 0.806). It finds relevant products whose titles share no word with the query, but tends to rank them lower. The hybrid is best on every metric, with gains of +0.029 nDCG@10 and +0.054 Recall@100 over BM25, and all of its differences from the single models are significant at $p < 0.001$.

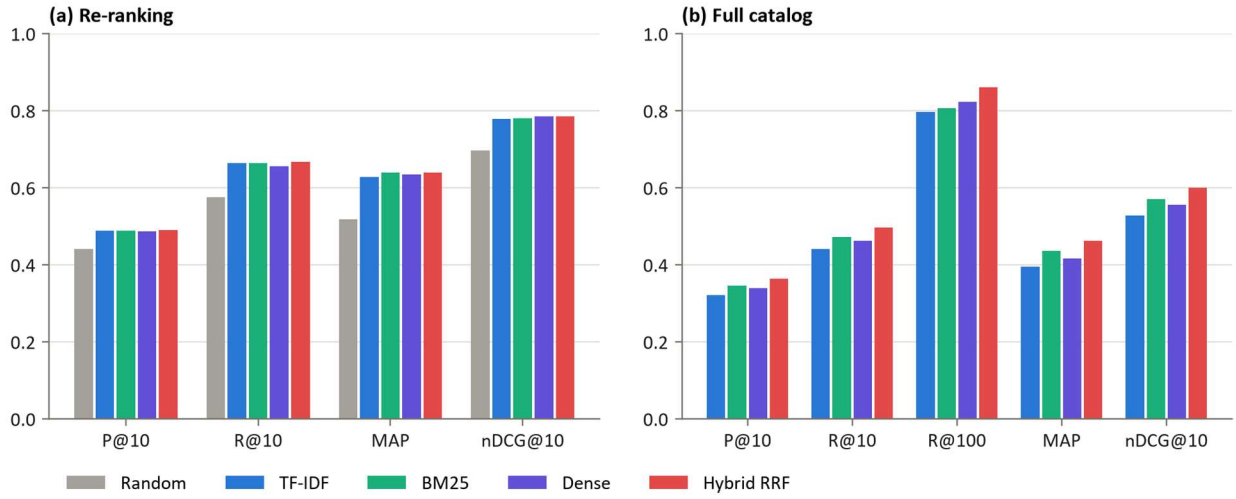


Figure 12: Test metrics under both protocols. Grey bars show a random ordering of the judged candidates.

The precision-recall curves in Figure 13 confirm the picture. In re-ranking the four curves nearly coincide and all lie well above the random curve. In retrieval the hybrid curve is highest at every recall level, and the gap to BM25 widens at higher recall, where the products found only by the dense model begin to count.

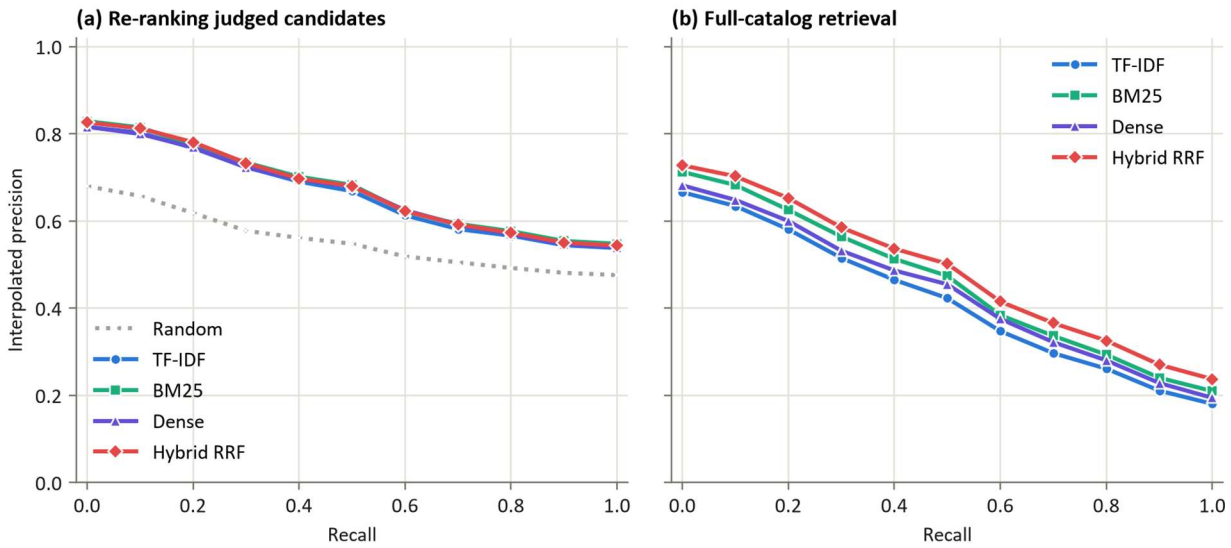


Figure 13: 11-point interpolated precision-recall curves on the test queries.

Table 8. Wilcoxon signed-rank p-values for per-query nDCG@10 on the test queries.

Pair	Re-ranking	Full catalog
TF-IDF vs BM25	0.204	< 0.001
TF-IDF vs Dense	0.265	< 0.001
TF-IDF vs Hybrid	0.003	< 0.001
BM25 vs Dense	0.654	0.041
BM25 vs Hybrid	0.100	< 0.001
Dense vs Hybrid	0.487	< 0.001

4.3 Tuning BM25

Figure 14 shows development-set nDCG@10 for 35 combinations of k1 and b. The best setting, k1 = 0.6 and b = 1.0, reaches a mean of 0.674 over the two protocols, against 0.670 for the common default of k1 = 1.2 and b = 0.75. Both values are plausible for product titles. A word rarely appears twice in a title, and when it does the repetition is usually keyword stuffing, so the term-frequency factor should saturate early. Long titles are mostly long lists of features, and full length normalisation keeps them from matching every query. The parameter b matters more in retrieval than in re-ranking. At k1 = 1.2, retrieval nDCG@10 rises from 0.532 at b = 0 to 0.560 at b = 1.

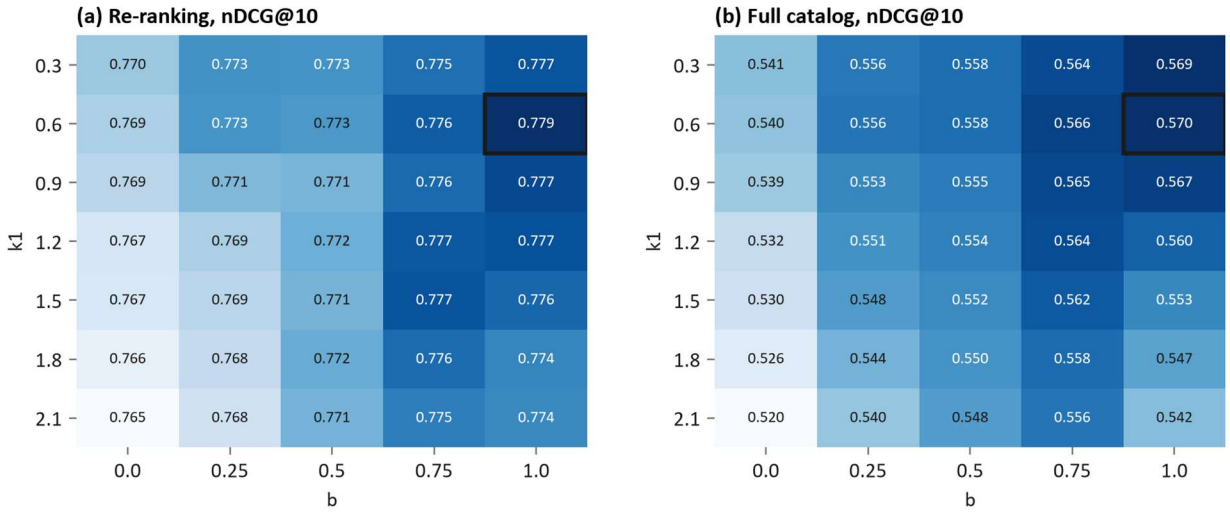


Figure 14: BM25 nDCG@10 on the development queries for each k1 and b. The outlined cell is the setting used in all other experiments.

4.4 Ablations on weighting and preprocessing

Figure 15a compares four TF-IDF weightings. For retrieval, logarithmic tf with idf is best (0.529 on the development queries), and removing idf costs 0.019. In re-ranking the four variants lie within 0.004 of each other, and the variant without idf is even slightly ahead, because all candidates already contain the main query words. This result changed our selection procedure. When settings were first chosen on re-ranking alone, the variant without idf was selected and then performed worst in retrieval. Settings are now chosen on the mean of both protocols.

Figure 15b removes one preprocessing step at a time. Stemming helps. Without it the vocabulary grows from 30,010 to 34,412 terms and BM25 retrieval drops from 0.570 to 0.530, since “shoe” no longer matches “shoes”. Stop-word removal had the opposite effect to the one we expected. Keeping the stop-words gives 0.574 for BM25 and 0.540 for TF-IDF, slightly above the standard pipeline. In shopping queries, words such as “for”, “with”, “no” and “in” often carry intent, as in “shoes for women”, “case with stand” or “2 in 1”. We kept the standard pipeline because the project specification requires stop-word removal. Removing both steps gives results close to removing stemming alone, which suggests that the two effects are largely independent.

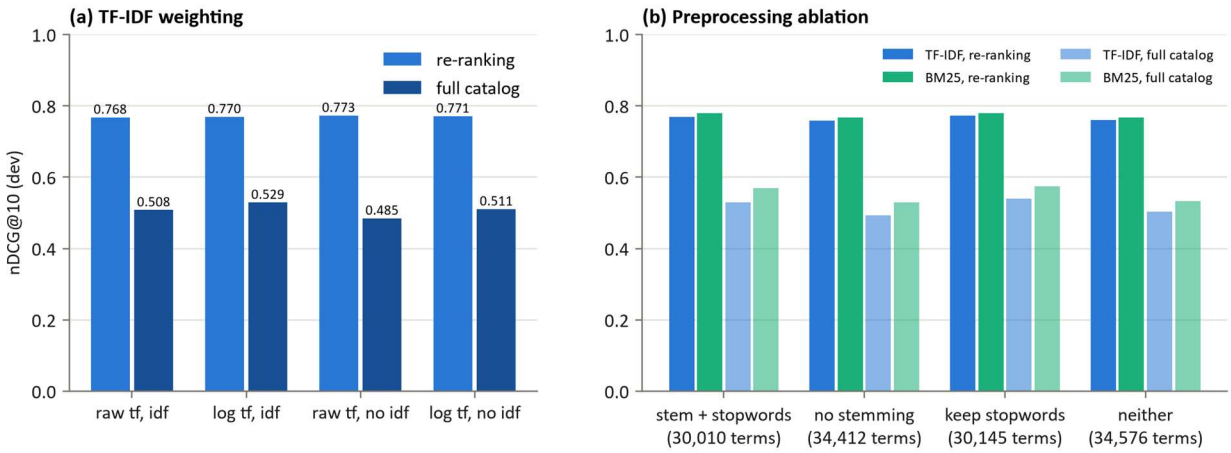


Figure 15: (a) TF-IDF weighting variants and (b) preprocessing ablation, nDCG@10 on the development queries. The number of index terms is given under each setting.

4.5 Fusion settings

The RRF constant has little influence (Figure 16a). Across k = 1 to 200, development nDCG@10 changes by at most 0.009 in either protocol. We kept k = 10, which had the best mean, although k = 60 would perform almost as well. The choice of lists matters far more (Figure 16b). BM25 with the dense model is the best pair. Adding TF-IDF as a third list lowers retrieval nDCG@10 from 0.596 to 0.579. TF-IDF and BM25 tend to make the same errors, and two votes for the same error outweigh the dense model. For the same reason the fusion of BM25 with TF-IDF alone leaves BM25 almost unchanged.

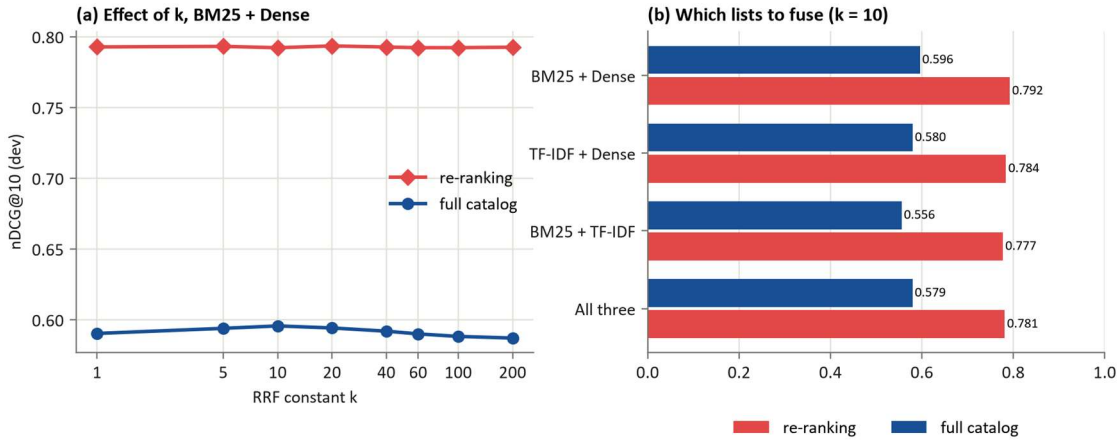


Figure 16: (a) RRF constant k and (b) choice of fused lists, nDCG@10 on the development queries.

4.6 Per-query analysis

Mean scores hide large disagreements on individual queries. Over the full catalog the dense model is better than BM25 on 313 test queries and worse on 339 (Figure 17a). The hybrid is better than BM25 on 377 queries and better than the dense model on 404 (Figure 17b). Table 9 lists typical cases, which are discussed below.

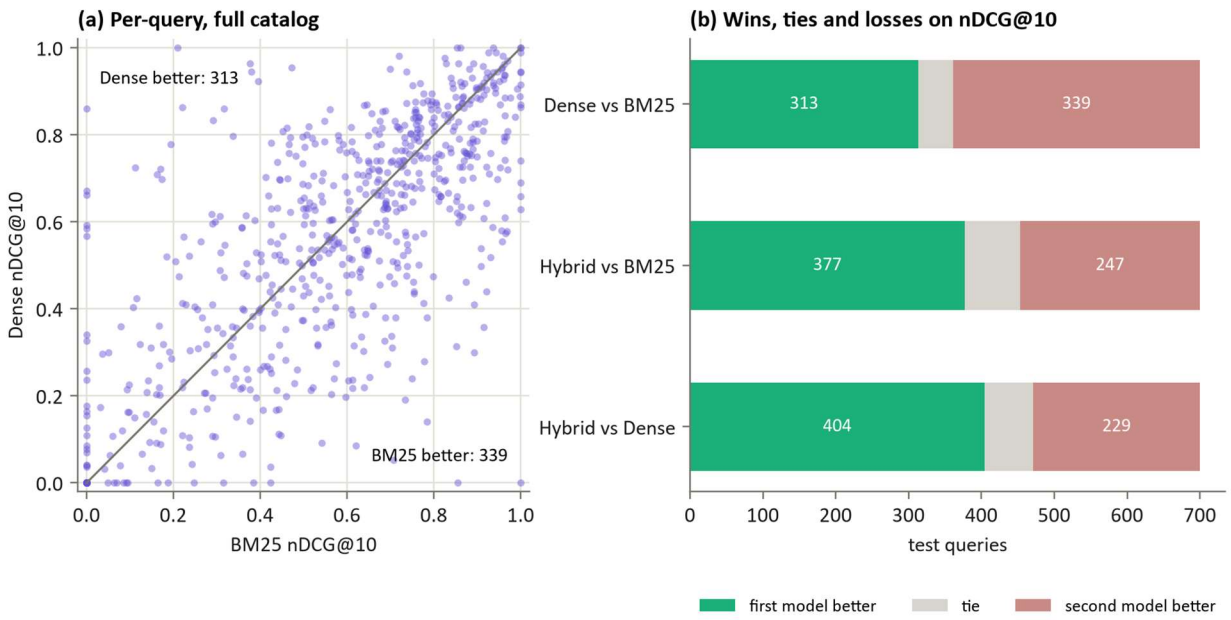


Figure 17: Per-query nDCG@10 in full-catalog retrieval. Points above the diagonal are queries on which the dense model is better than BM25.

BM25 performs better when the query names something that must appear in the title word for word. This may be a colour ("brown flats for women"), a size ("14 gallon trash can") or an uncommon brand or product name ("tidy die", "acudraw pro"). The dense model recognises the product type, shoes or bins, but treats the colour or size as optional. The dense model performs better on spelling variants and on words written together or apart, none of which match a stem in the index. Titles write "GoPro", not "go pro", and "ashwaghanda" is a misspelling of ashwagandha. In total 10.6% of the test queries contain at least one stem that does not occur in the index. On these queries BM25 averages 0.377 nDCG@10 and the hybrid 0.422.

Table 9. Queries with a large gap between BM25 and the dense model (nDCG@10, full catalog).

Query	BM25	Dense	Hybrid
brown flats for women	0.86	0.00	0.52
14 gallon trash can	0.78	0.28	0.80
tidy die	1.00	0.00	1.00
acudraw pro	0.75	0.24	0.65
go pro camera	0.00	0.86	0.50
ashwaghanda extract	0.38	0.96	0.86
hinatab	0.00	0.66	0.35
playstation 1	0.16	0.71	0.39

Figure 18 breaks the results down by query length. In retrieval, one- and two-word queries are the hardest for every model because they leave the intent open. In re-ranking the longest queries are the

hardest, as they usually describe one specific item that few candidates match in every detail. In retrieval the hybrid is ahead in every length group.

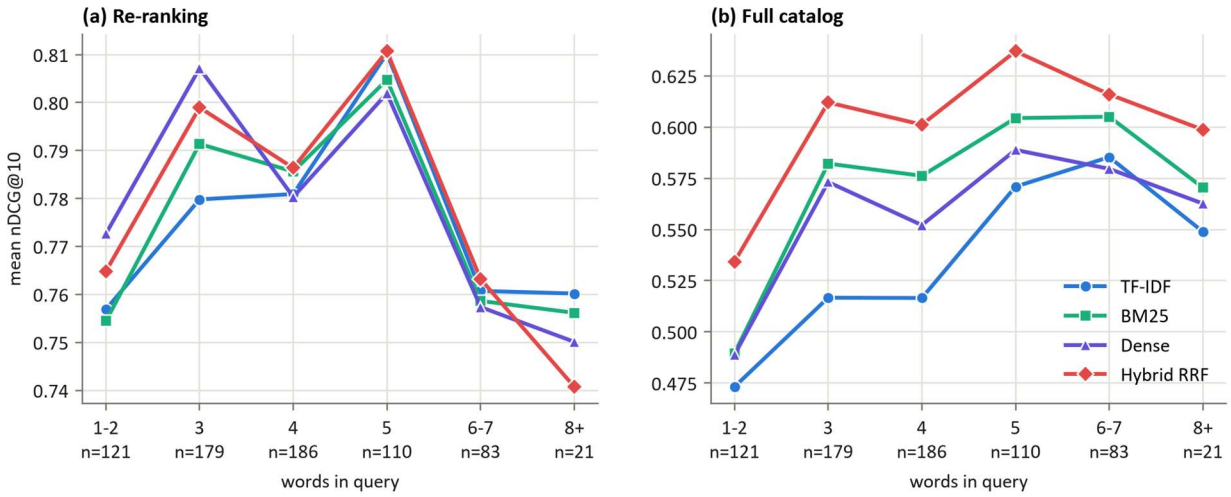


Figure 18: Mean nDCG@10 by number of words in the query (n is the number of test queries).

Figure 19 shows which labels appear at each rank position when re-ranking. Under a random ordering about 45% of the products at any position are Exact. The four models raise this to 62%–66% at rank 1, and the share of Exact products then declines steadily down the list. The share of Irrelevant products at rank 1 falls from 18% to between 8% and 10%.

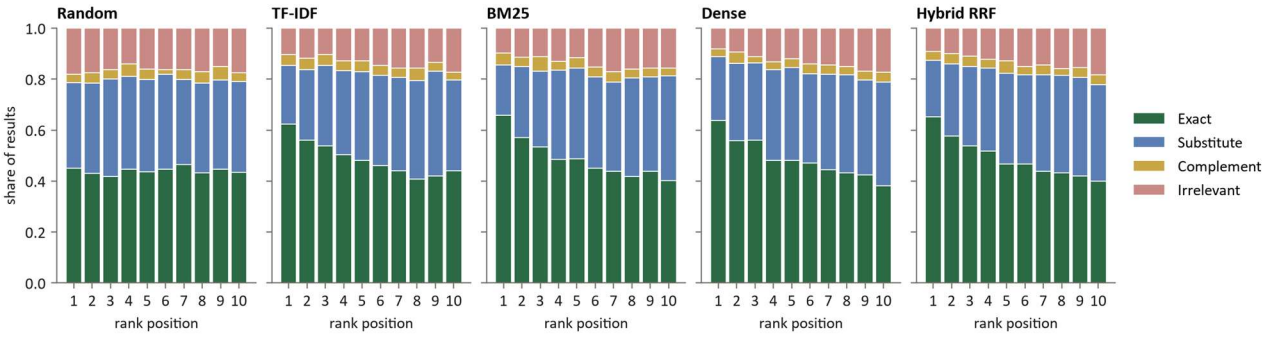


Figure 19: Label mix at rank positions 1 to 10 when re-ranking the judged candidates.

4.7 Computational cost

The lexical models answer a query over the full catalog in about 0.3 ms (Figure 20). The dense model needs 13 ms, nearly all of it spent encoding the query, and the hybrid, which also runs BM25, needs 14 ms. Our first timing run reported the hybrid as faster than the dense model alone, which cannot be correct. The dense model had been timed first and was paying the cost of loading the encoder, so every model now receives a short warm-up before it is timed. Building the inverted index takes 6.6 s, and encoding the catalog, which is done once, took about a minute and a half. All of these times are acceptable for an interactive search page. The gap would matter more on a larger catalog, since the dense model computes a dot product with every product vector, while BM25 reads only the postings of the query terms.

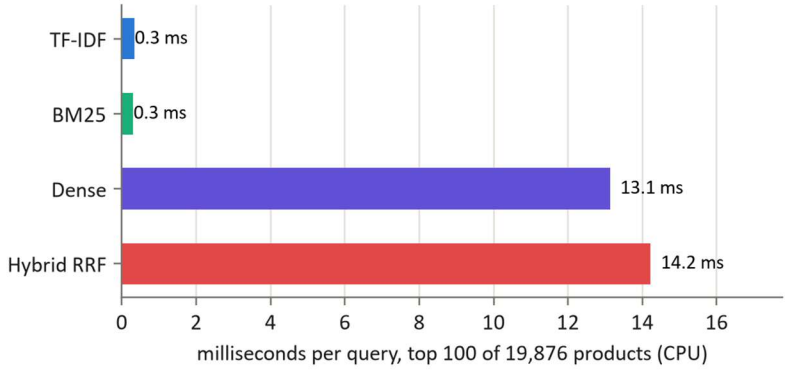


Figure 20: Average query time for the top 100 products over the full catalog.

4.8 Search interface

Figures 21 to 24 show the interface. On the search page (Figure 21), TF-IDF and BM25 results show the contribution of each query stem to the score. A side panel shows the query after cleaning, tokenization and stemming together with the document frequency and idf of each stem. For the hybrid (Figure 22) each result shows the BM25 and dense ranks that were fused. When the query belongs to the benchmark, each result carries its E/S/C/I judgment and the page reports nDCG@10 for the list. This is how the “In the Woods” result described in Section 4.2 was found. The comparison page (Figure 23) runs one query through all four models side by side, and two further pages present the evaluation results and the index statistics (Figure 24).

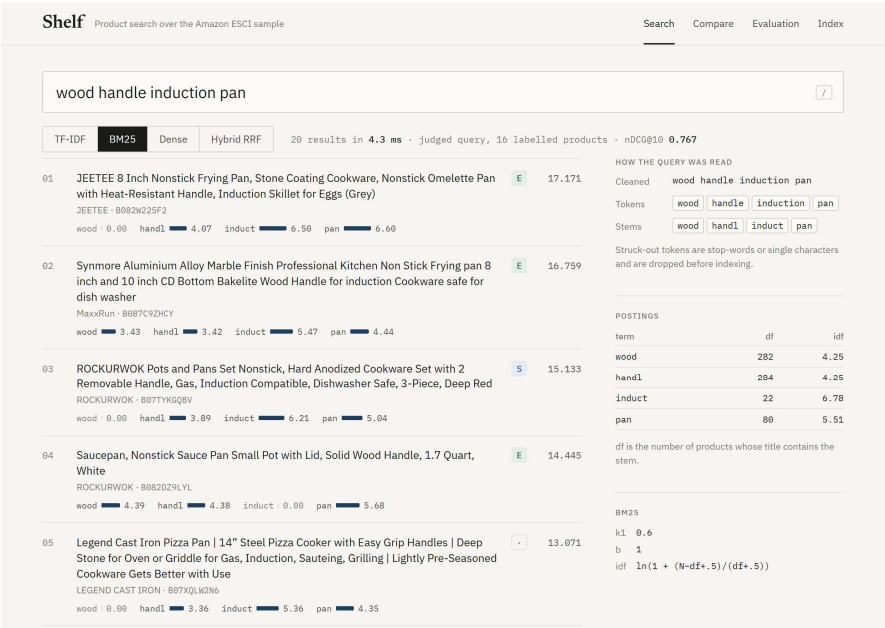


Figure 21: Search page with BM25. The bars under each result are per-stem BM25 contributions; the letters are ESCI judgments.

The hybrid view makes the fusion easy to follow. For “go pro camera” the first fused result is a helmet mount ranked first by BM25 and seventeenth by the dense model. The GoPro HERO7 camera, an Exact product, is only 46th in BM25 but first in the dense ranking, and fusion places it second.

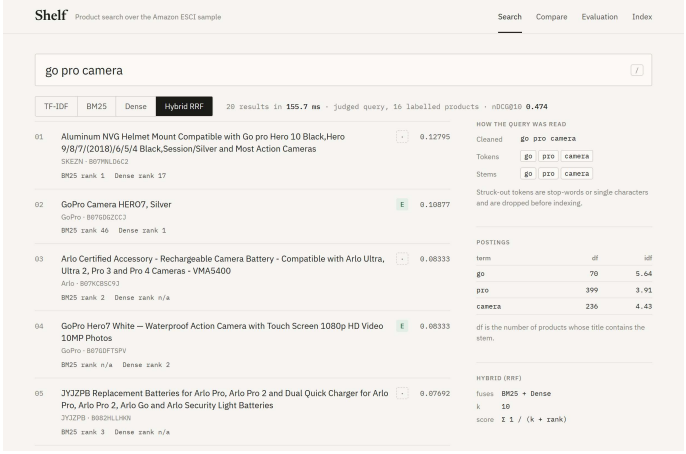


Figure 22: Search page with the hybrid model for “go pro camera”, showing the BM25 and dense ranks behind each fused result.

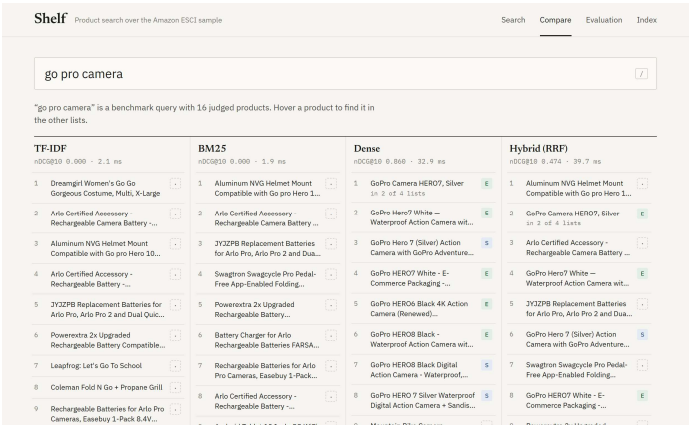


Figure 23: Comparison page for “go pro camera”. Both lexical models score 0 because the catalog writes “GoPro”, while the dense model finds the cameras.

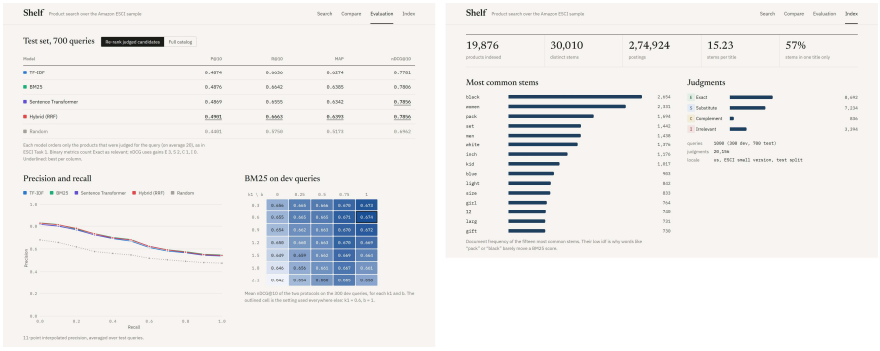


Figure 24: (a) Evaluation page with the test results, precision-recall curves and the BM25 parameter grid. (b) Index page with vocabulary and postings size, the most frequent stems and the label distribution.

5. Conclusion

This project implemented four retrieval models for product search on a common preprocessing pipeline and inverted index, and compared them on 1,000 queries from the Amazon ESCI dataset under two evaluation protocols. The answers to the two questions posed in the introduction are fairly clear.

On product titles a tuned BM25 is a strong baseline. It matches the pretrained sentence encoder on average, runs about 42 times faster, and every score it produces can be explained term by term. The dense model is not a replacement for it, but it is a useful complement. It is better on misspelled and compound queries and finds relevant products that share no words with the query, while it is worse at enforcing colours, sizes and brand names. Because the two models fail on different queries, Reciprocal Rank Fusion produces a clear improvement without any training. Over the full catalog the hybrid reaches 0.600 nDCG@10 against 0.570 for BM25, and 0.860 Recall@100 against 0.806, for a cost of about 14 ms per query on a laptop CPU. In our view this is worth paying.

The re-ranking task gave a different result. All four models land close together, and not far above a random ordering. Amazon's engine had already selected the candidates, and the remaining distinction between Exact and Substitute often depends on information that is not in the title.

Several lessons concern the evaluation rather than the models. Counting Substitute products as relevant made every model look good and hid the differences between them. Selecting settings on one protocol alone chose a TF-IDF variant that was the worst on the other. Timing measurements were distorted by model loading until a warm-up was added. Each of these problems was visible only because the evaluation was built to report both protocols, a random baseline and per-query results.

The study has limitations. It uses 1,000 queries and product titles only, and descriptions or bullet points would give every model more text to match. Treating unjudged products as irrelevant underestimates all models in retrieval, probably the dense model most, since it retrieves products worded differently from the ones the annotators saw. The dense encoder was used as released, without fine-tuning on ESCI.

Four extensions follow directly from the results. The first is to keep intent-carrying stop-words such as "for" and "with" in the index, which the ablation suggests would improve both lexical models. The second is to split or join compound tokens such as "gopro". The third is to fine-tune MiniLM on the ESCI training split. The last is to re-rank the top fused candidates with a cross-encoder. Such a model reads the query and the title together and may be able to separate Exact from Substitute products where the present models cannot.

References

1. Robertson, S., Zaragoza, H.: The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval 3(4), 333–389 (2009)

2. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: Proceedings of EMNLP-IJCNLP, pp. 3982–3992 (2019)

3. Cormack, G.V., Clarke, C.L.A., Buettcher, S.: Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods. In: Proceedings of SIGIR, pp. 758–759 (2009)

4. Reddy, C.K., Màrquez, L., Valero, F., Rao, N., Zaragoza, H., Bandyopadhyay, S., Biswas, A., Xing, A., Subbian, K.: Shopping Queries Dataset: A Large-Scale ESCI Benchmark for Improving Product Search. arXiv:2206.06588 (2022)

5. Salton, G., Buckley, C.: Term-weighting approaches in automatic text retrieval. Information Processing & Management 24(5), 513–523 (1988)

6. Porter, M.F.: An algorithm for suffix stripping. Program 14(3), 130–137 (1980)

7. Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., Zhou, M.: MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In: Advances in NeurIPS 33 (2020)

8. Järvelin, K., Kekäläinen, J.: Cumulated gain-based evaluation of IR techniques. ACM Transactions on Information Systems 20(4), 422–446 (2002)

9. Manning, C.D., Raghavan, P., Schütze, H.: Introduction to Information Retrieval. Cambridge University Press (2008)

10. Wilcoxon, F.: Individual comparisons by ranking methods. Biometrics Bulletin 1(6), 80–83 (1945)